ARM Cortex-M
External Interrupts (EXTI)

CSE 4095
Special Topics in Computer Engineering III:
Introduction to Embedded Systems
Spring 2023

Taken from Dr. Yifeng Zhu's Lecture Notes
Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C – Chapter 11

Interrupt *vs* Polling



- **Polling**:

You pick up the phone every few seconds to check whether you are getting a call.

Interrupt *vs* Polling



http://www.istockphoto.com/

- **Polling**:

You pick up the phone every few seconds to check whether you are getting a call.

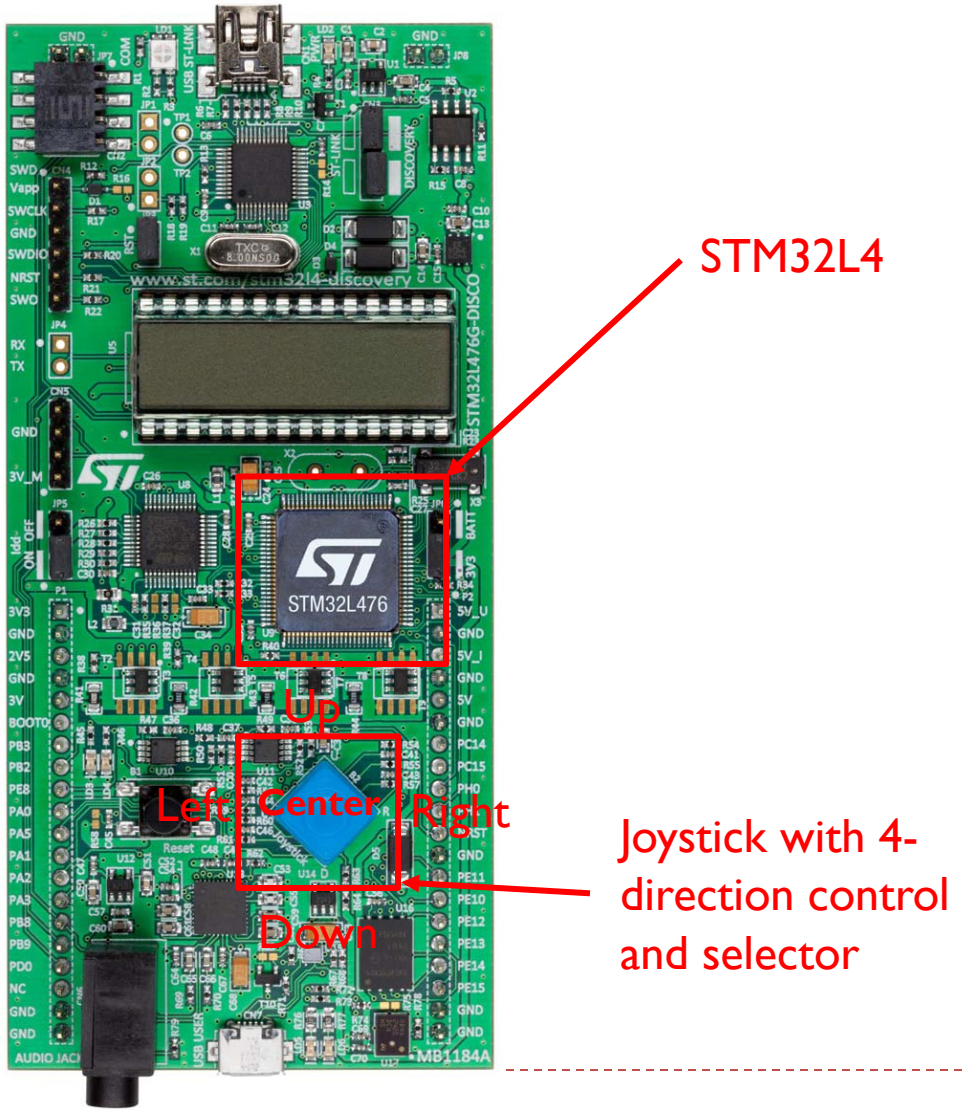
- **Interrupt**:

Do whatever you should do and pick up the phone when it rings.

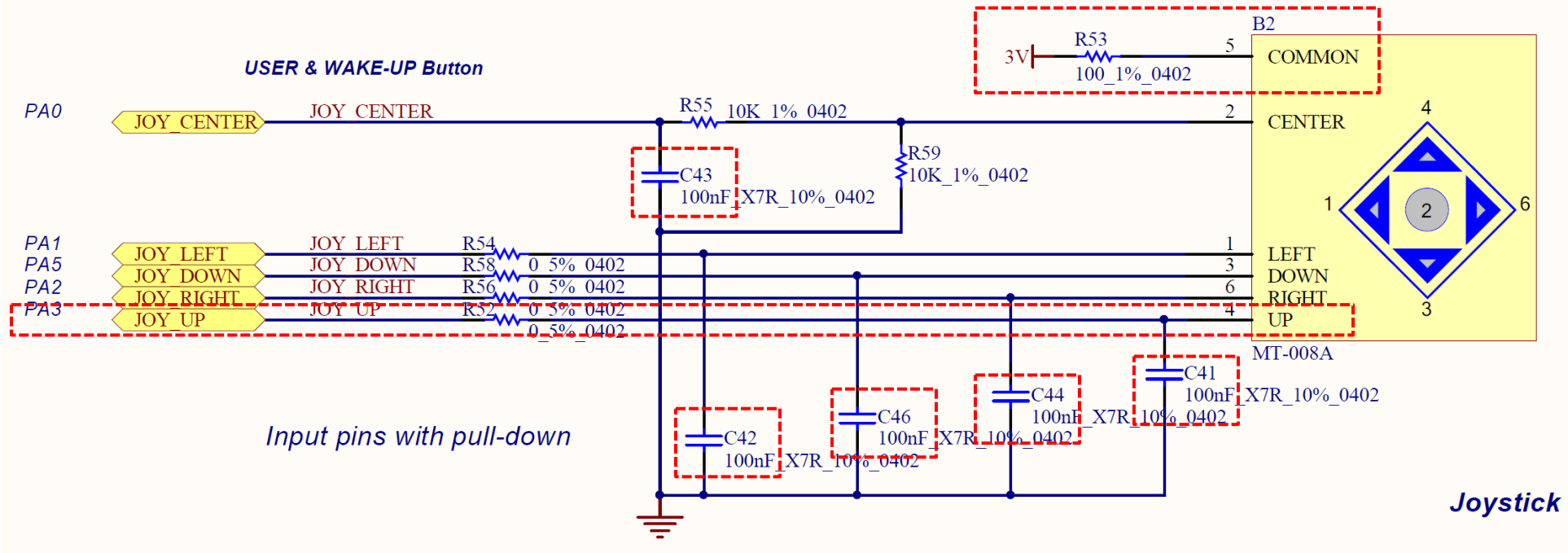
  - Hardware-triggered software action
  - Respond to external events efficiently, while avoiding busy wait.
  - When no events, the processor can run the main program or enter a sleep state to conserve energy

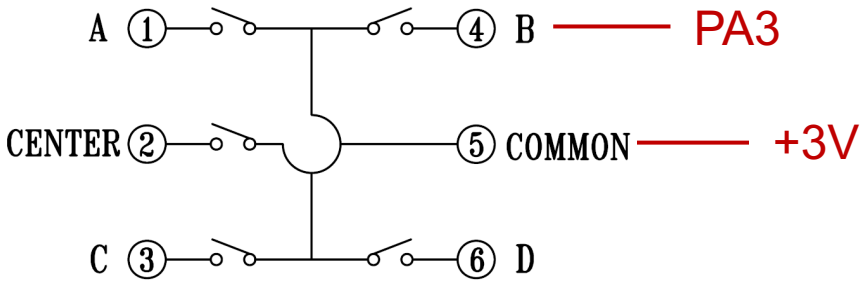
Joystick

STM32L4 Discovery Kit



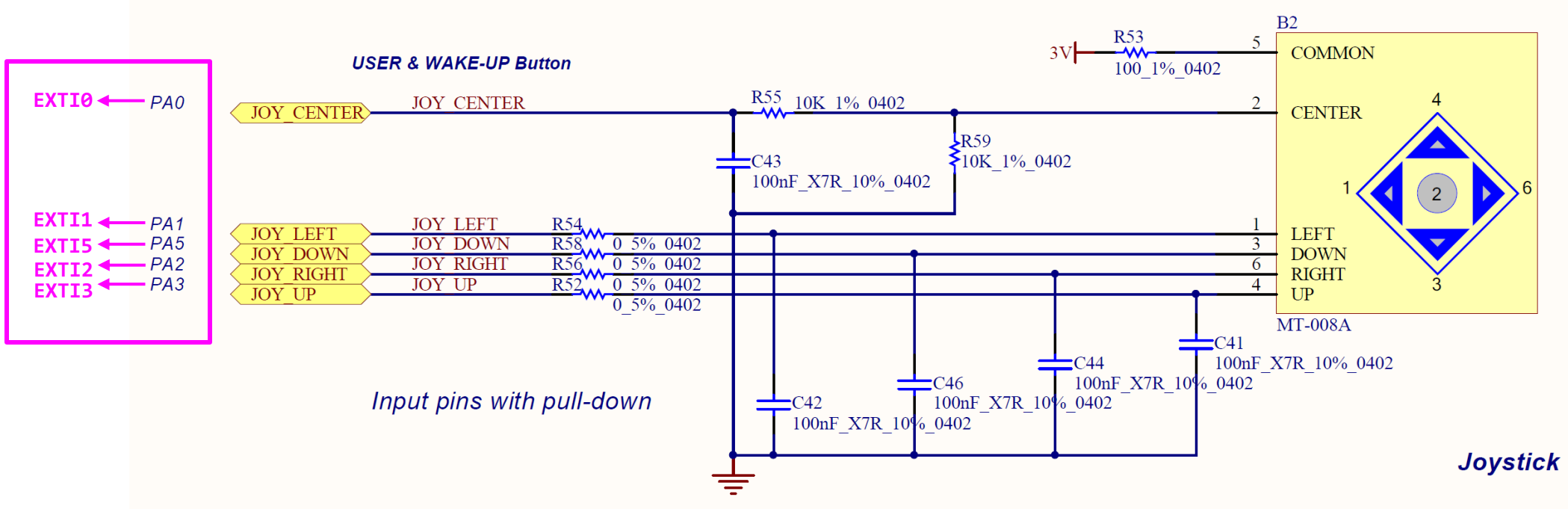
Joystick

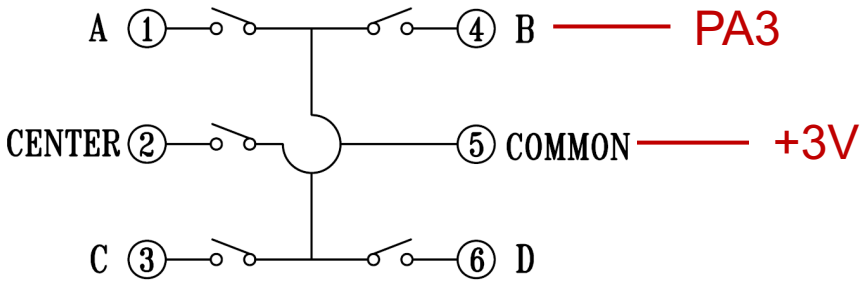




Internal diagram of joystick

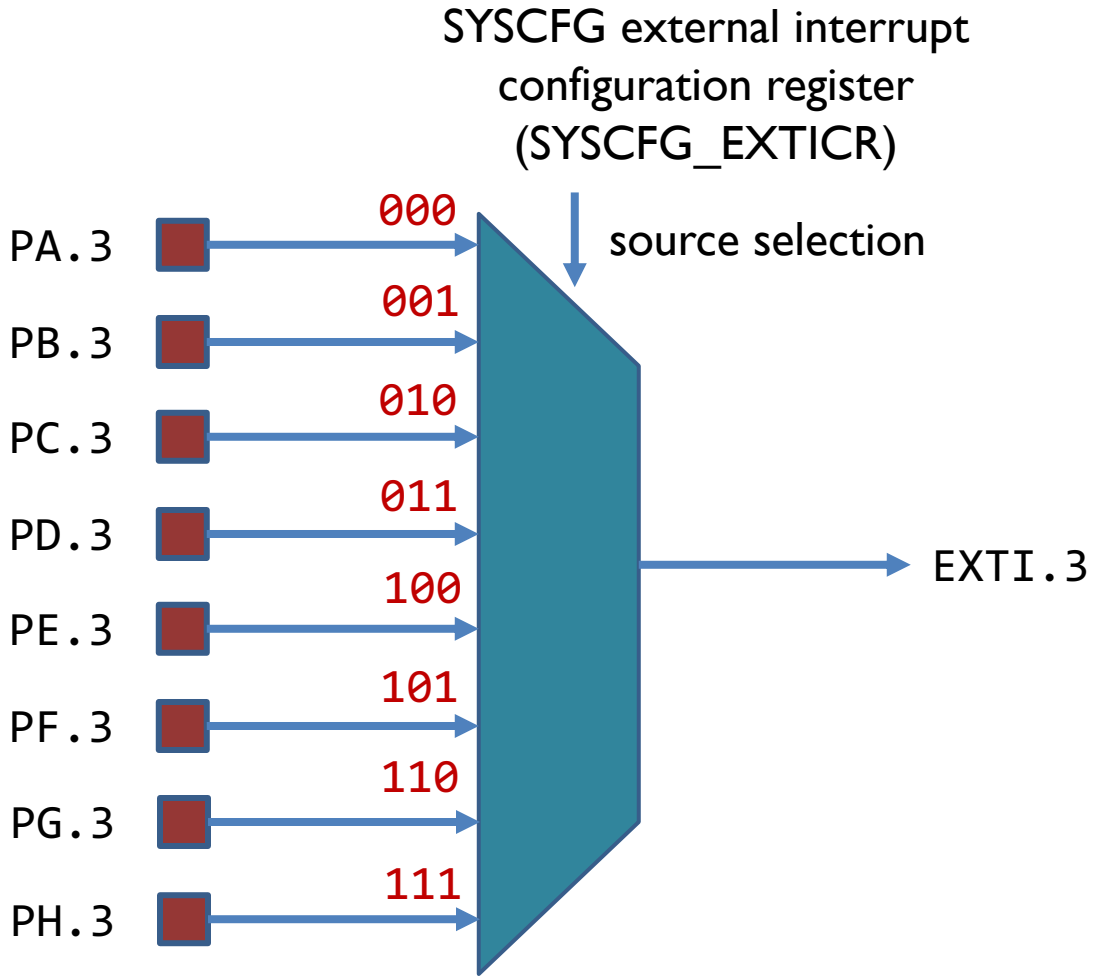
Joystick



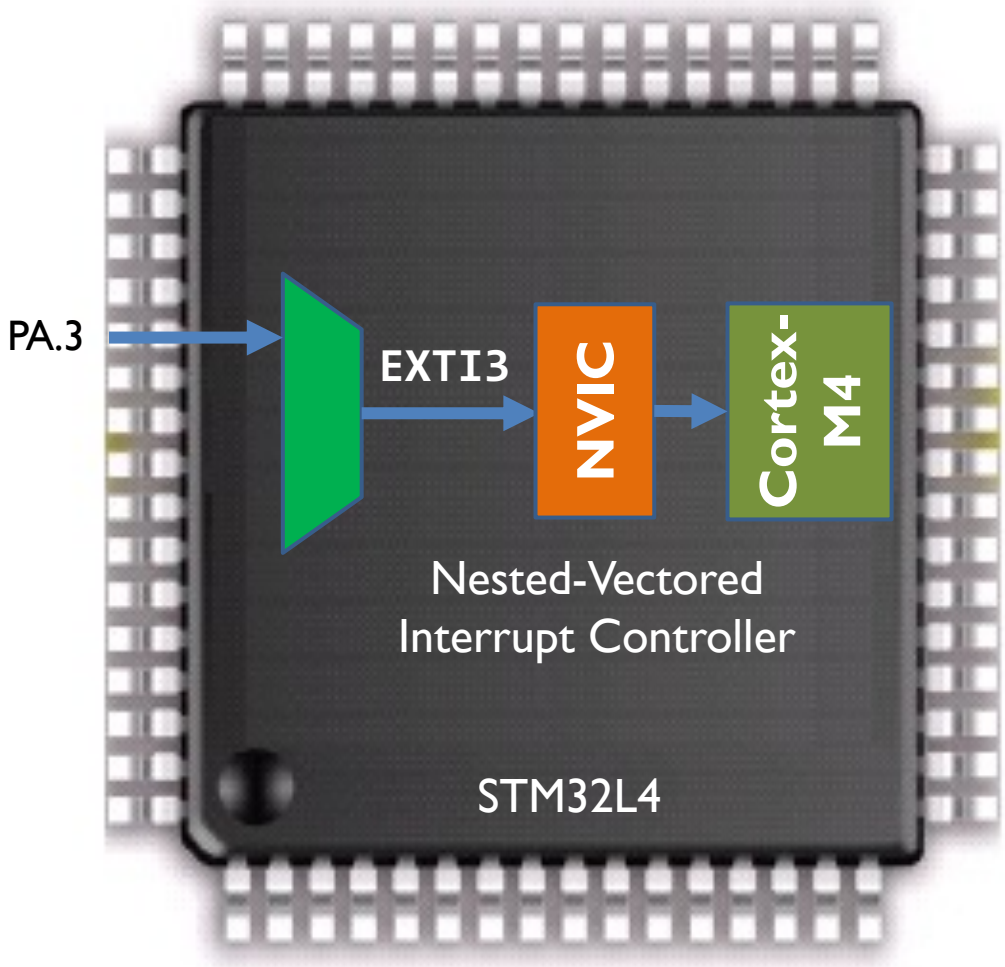


Internal diagram of joystick

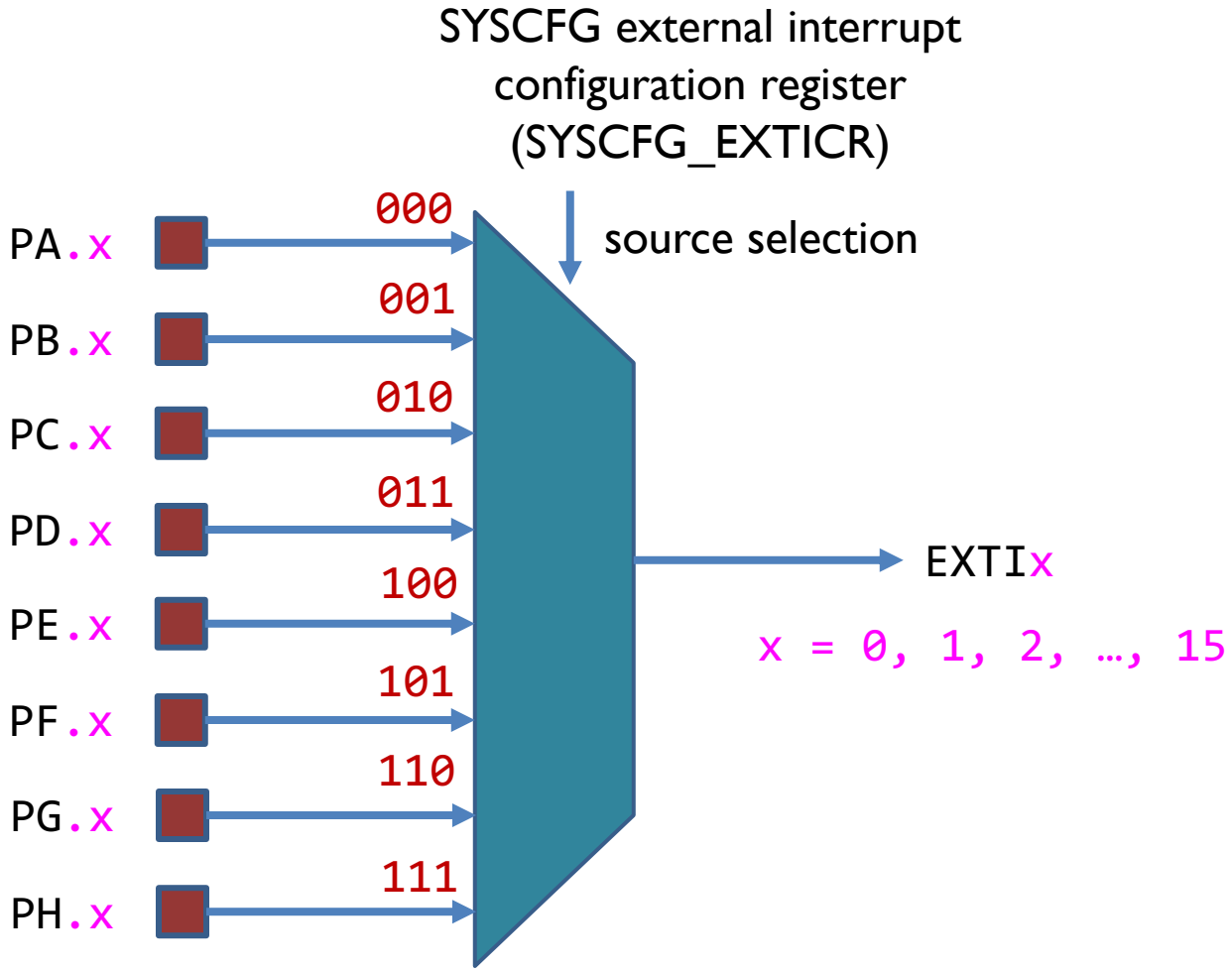
External Interrupt (EXTI) Sources

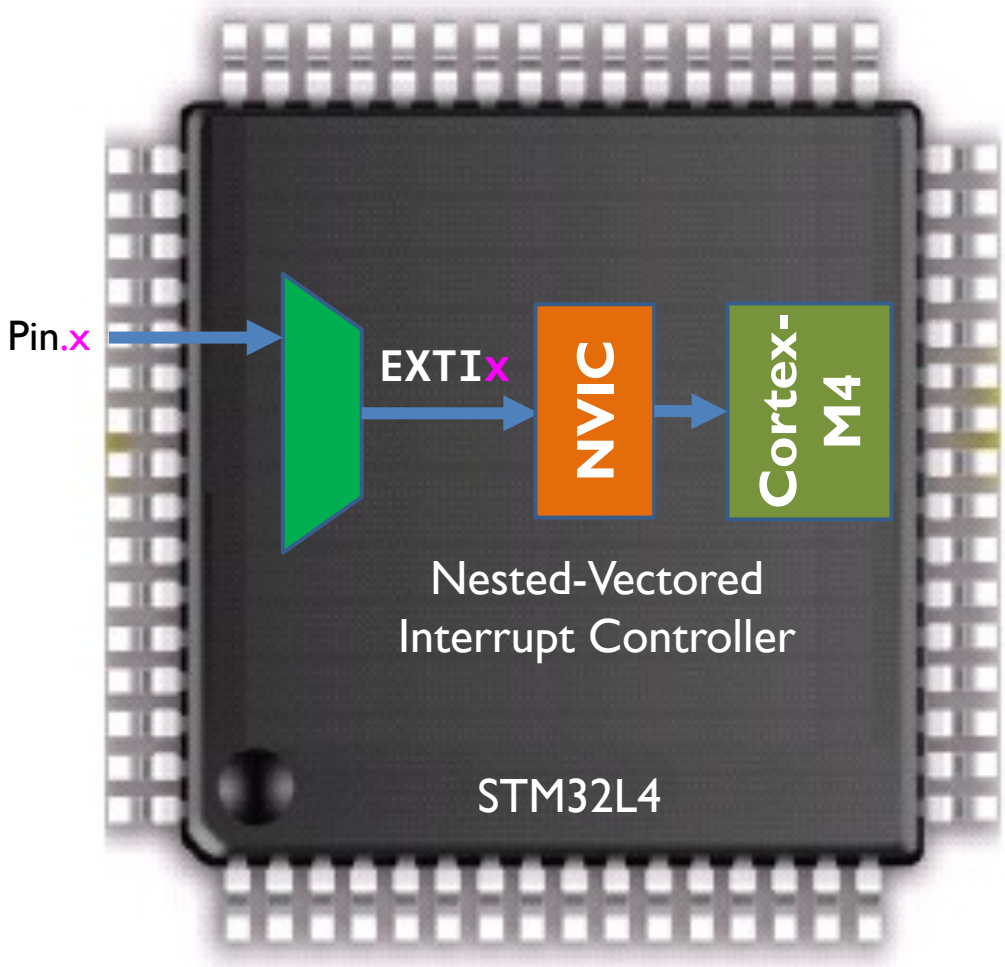


Select pin x from Port y as EXTIx



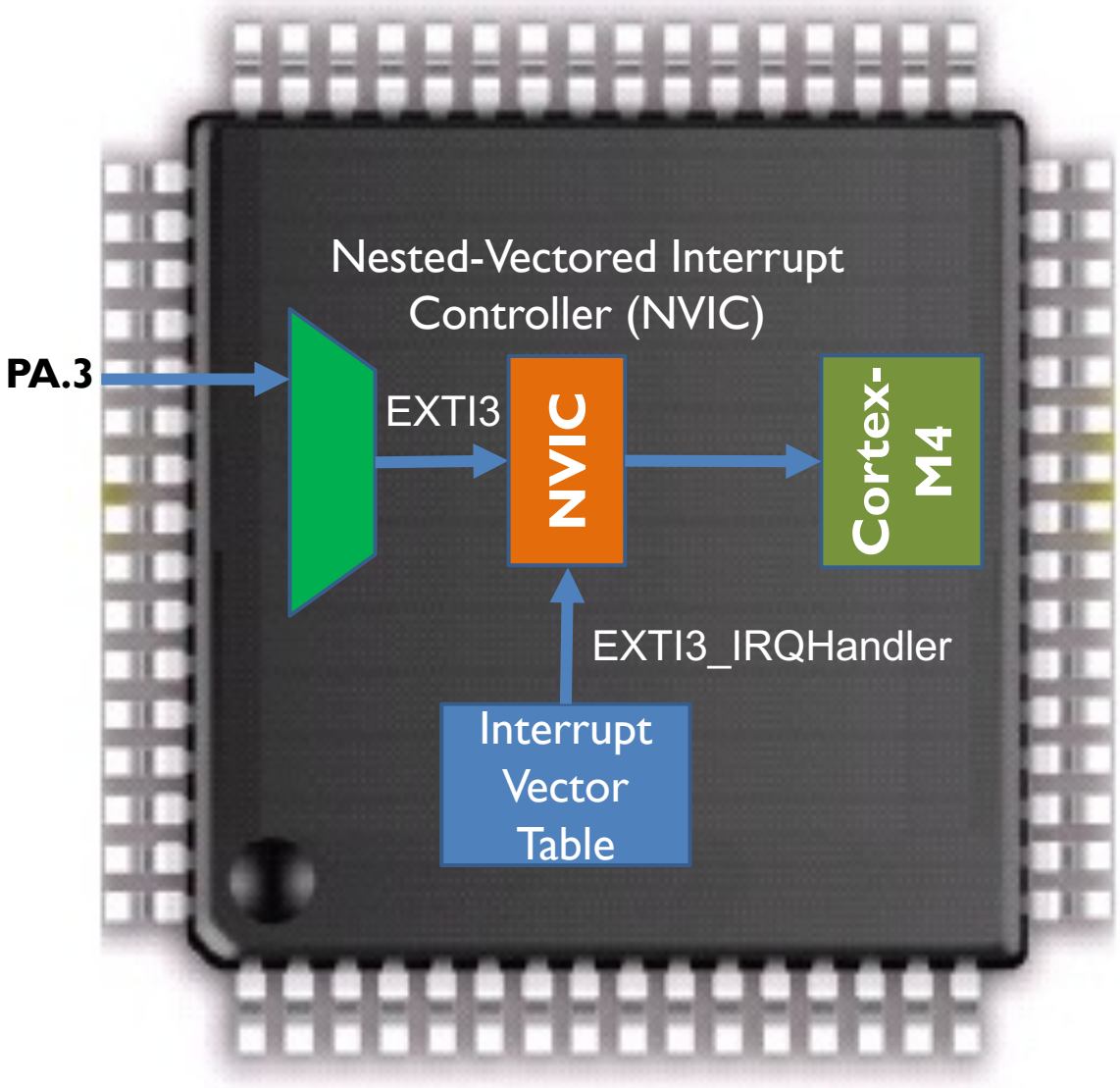
External Interrupt (EXTI) Sources





One mux for each of EXTI0, EXTI1, ..., EXTI15

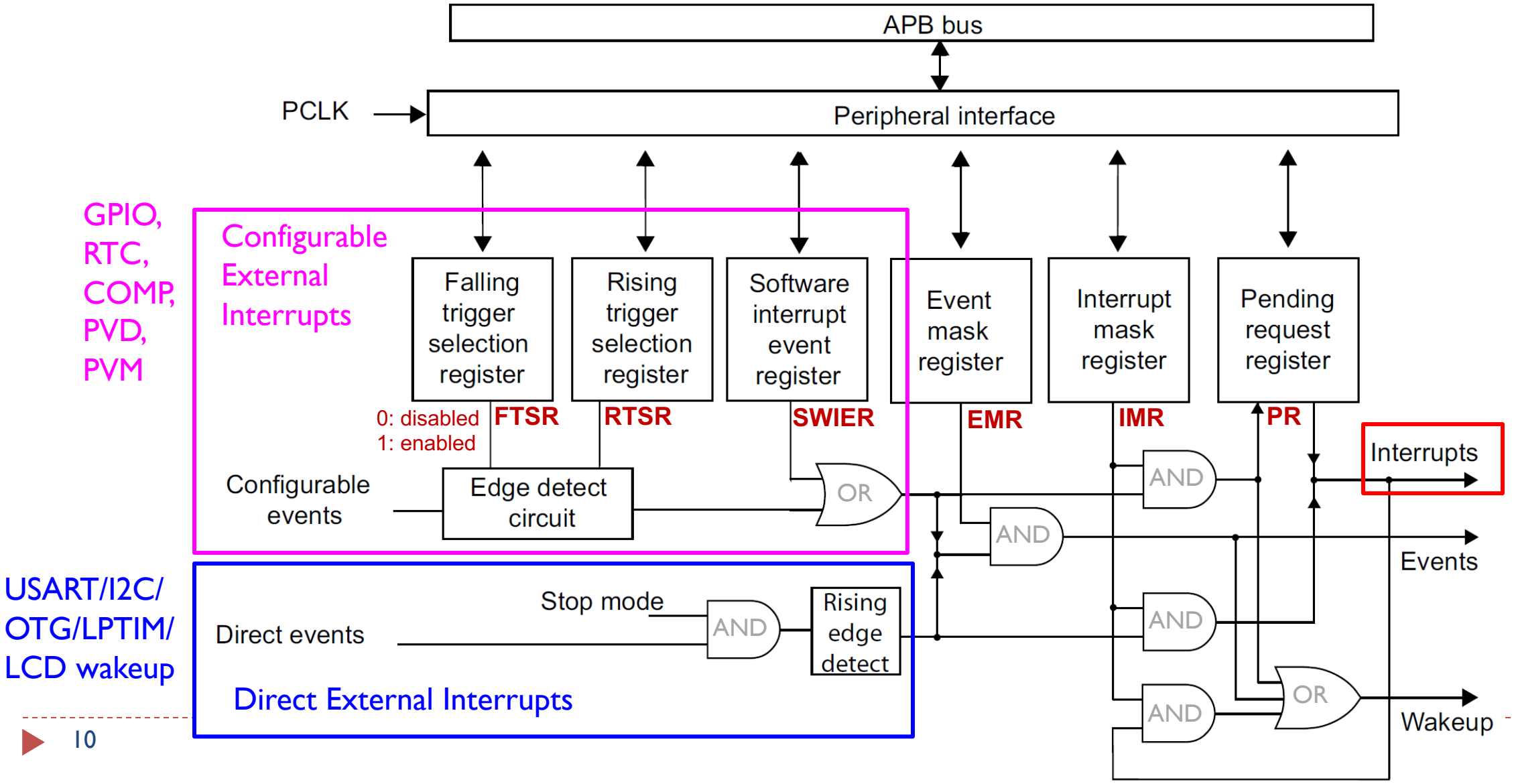
Interrupt Vector Table



Interrupt Vector Table

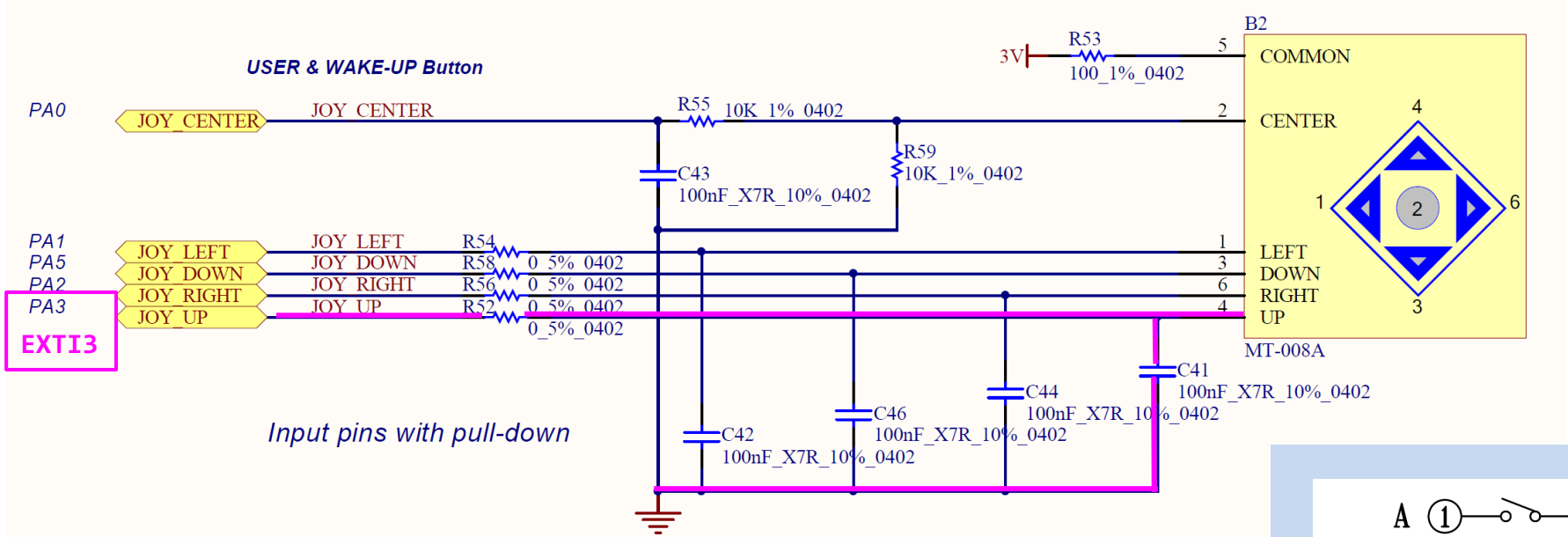
Interrupt Number (8 bits)	Memory Address of ISR (32 bits)
...	...
EXTI0_IRQn = 6	EXTI0_IRQHandler
EXTI1_IRQn = 7	EXTI1_IRQHandler
EXTI2_IRQn = 8	EXTI2_IRQHandler
EXTI3_IRQn = 9	EXTI3_IRQHandler
EXTI4_IRQn = 10	EXTI4_IRQHandler
...	...
EXTI9_5_IRQn = 23	EXTI9_5_IRQHandler
...	...
EXTI15_10_IRQn = 40	EXTI15_10_IRQHandler
...	...

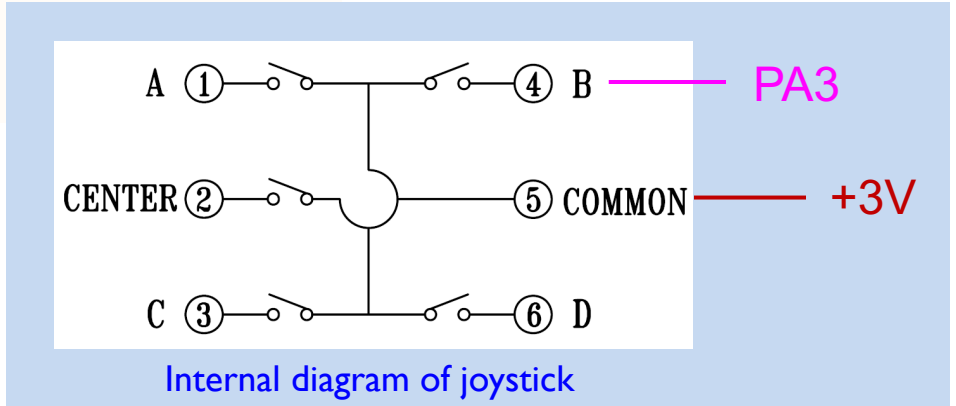
External Interrupt (EXTI) Controller



STM32L4 Discovery Kit

Software goal: If the up button is pressed, toggle LED.





Internal diagram of joystick

Note: PA3 is neither pulled up nor pulled down externally.

```
//  Pull PA.3 down internally; Trigger on rising edge
int main(void) {
  // Enable GPIO Clock
  RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;




}
```

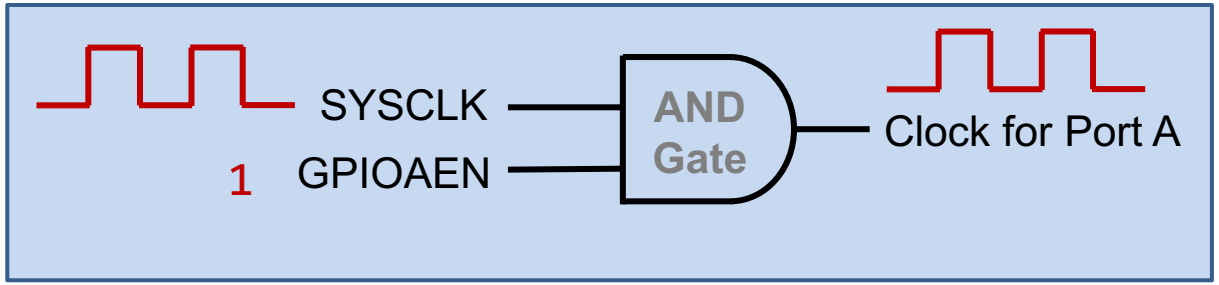
```
//stm32l476xx.h
#define  RCC_AHB2ENR_GPIOAEN ((uint32_t)0x00000001U)
```

AHB2 peripheral clock enable register (AHB2ENR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RNG EN	Res.	AESEN
													rw		rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	ADCEN	OTGFS EN	Res.	Res.	Res.	Res.	GPIOH EN	GPIOG EN	GPIOF EN	GPIOE EN	GPIOD EN	GPIOC EN	GPIOB EN	GPIOA EN
		rw	rw					rw	rw	rw	rw	rw	rw	rw	rw

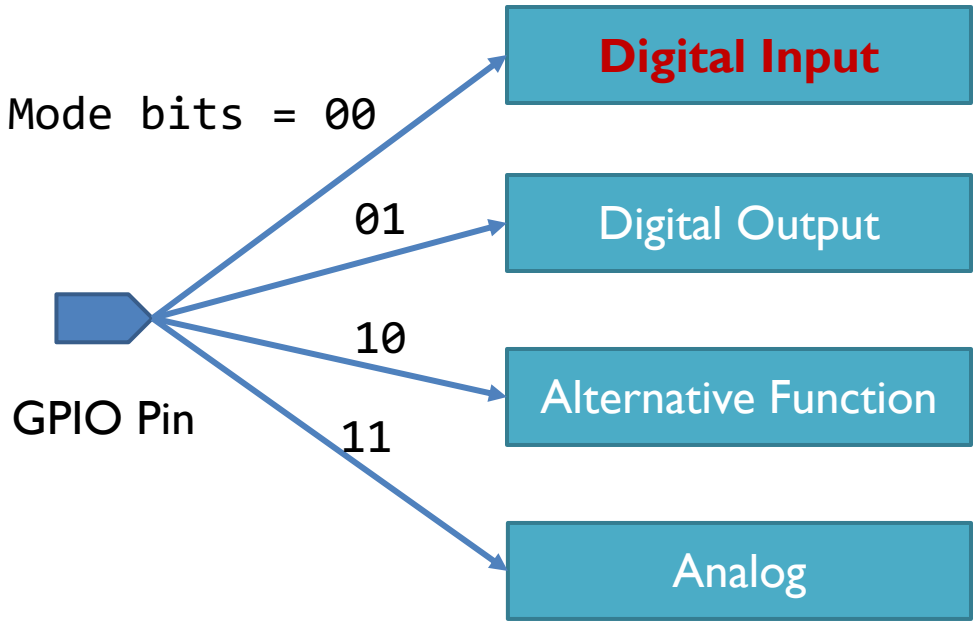
1



```
//  Pull PA.3 down internally; Trigger on rising edge
int main(void) {
  // Enable GPIO Clock
  RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;
  // GPIO Mode: Input(00), Output (01),
  //            AF(10),    Analog (11, default)
  GPIOA->MODER &= ~3U << 6;



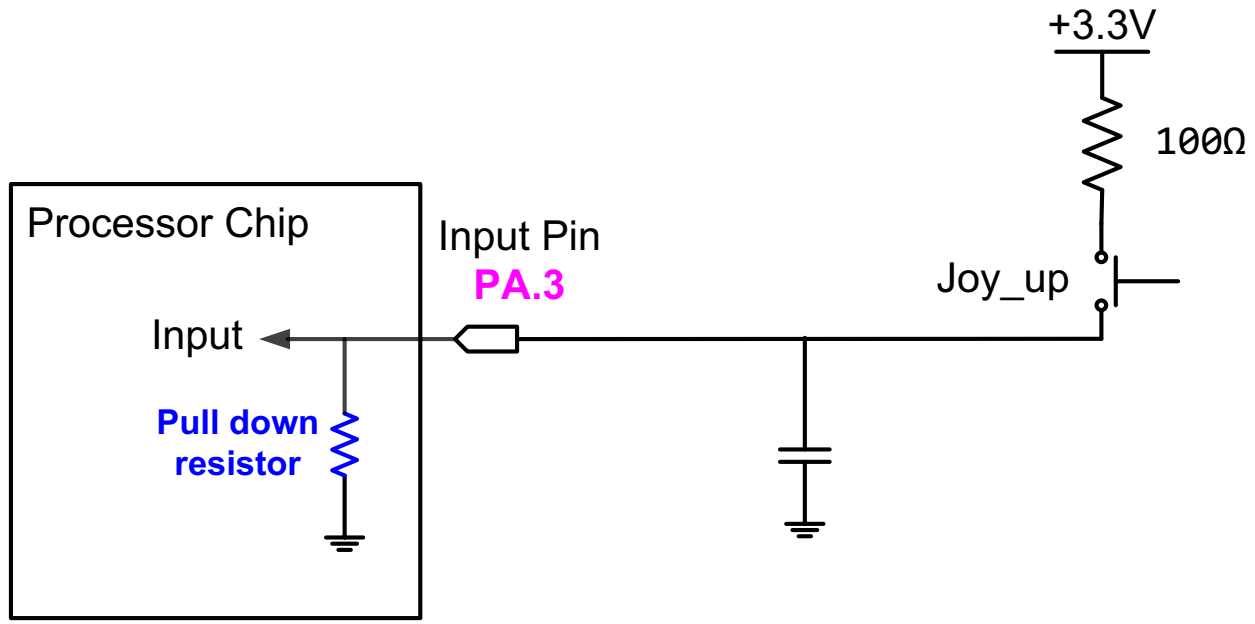

}
```



```
//  Pull PA.3 down internally; Trigger on rising edge
int main(void) {
  // Enable GPIO Clock
  RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;
  // GPIO Mode: Input(00), Output (01),
  //            AF(10),    Analog (11)
  GPIOA->MODER &= ~3U << 6;
  // GPIO Push-Pull: No pull-up, pull-down (00),
  //  Pull-up (01), Pull-down (10), Reserved (11)
  GPIOA->PUPDR &= ~3U << 6;
  GPIOA->PUPDR |= 2U << 6;     // Pull down




}
```

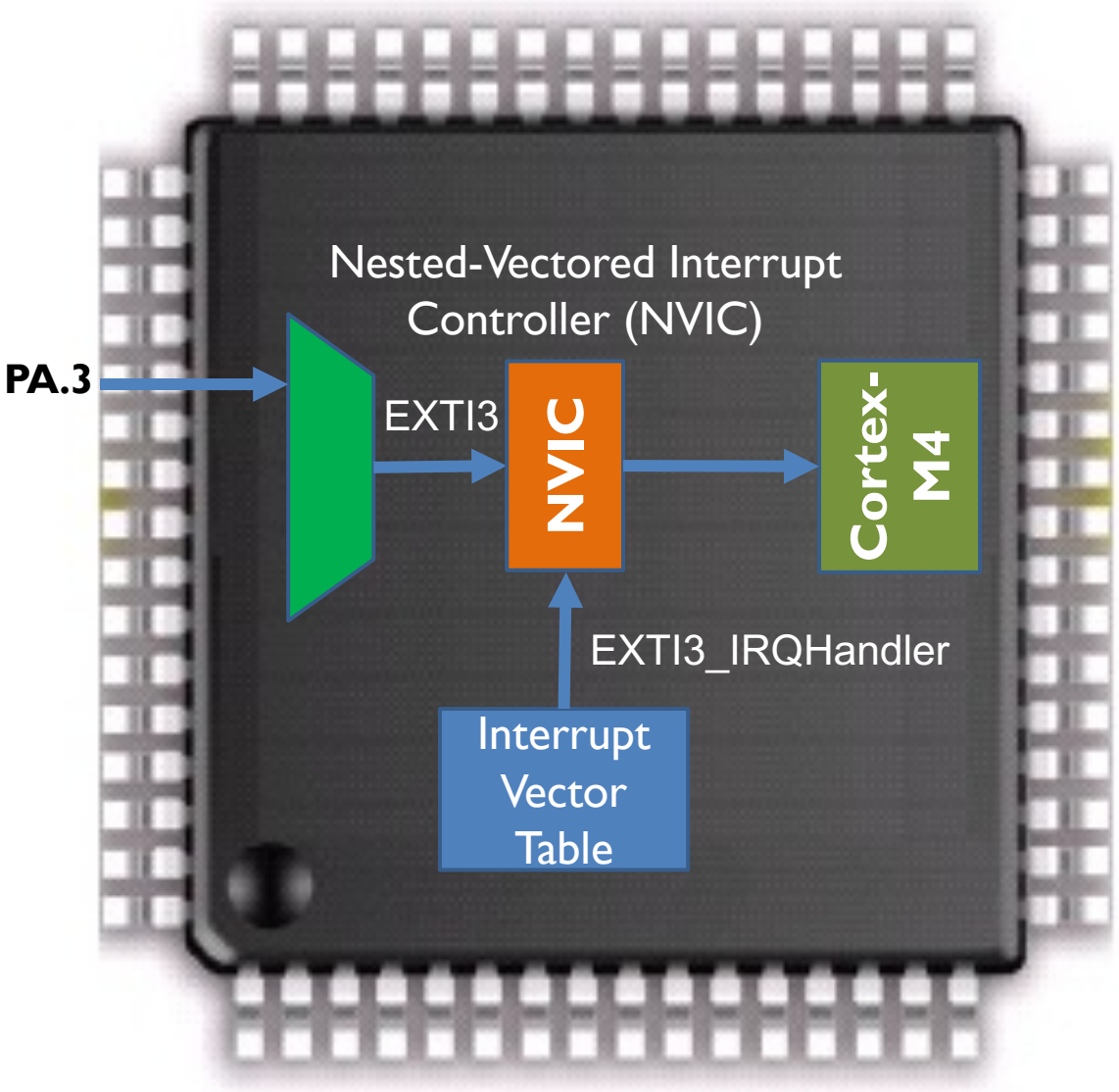


```
//  Pull PA.3 down internally; Trigger on rising edge
int main(void) {
  // Enable GPIO Clock
  RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;
  // GPIO Mode: Input(00), Output (01),
  //            AF(10),    Analog (11)
  GPIOA->MODER &= ~3U << 6;
  // GPIO Push-Pull: No pull-up, pull-down (00),
  //  Pull-up (01), Pull-down (10), Reserved (11)
  GPIOA->PUPDR &= ~3U << 6;
  GPIOA->PUPDR |= 2U << 6;     // Pull down

  NVIC_EnableIRQ(EXTI3_IRQn); // Enable Interrupt




}
```



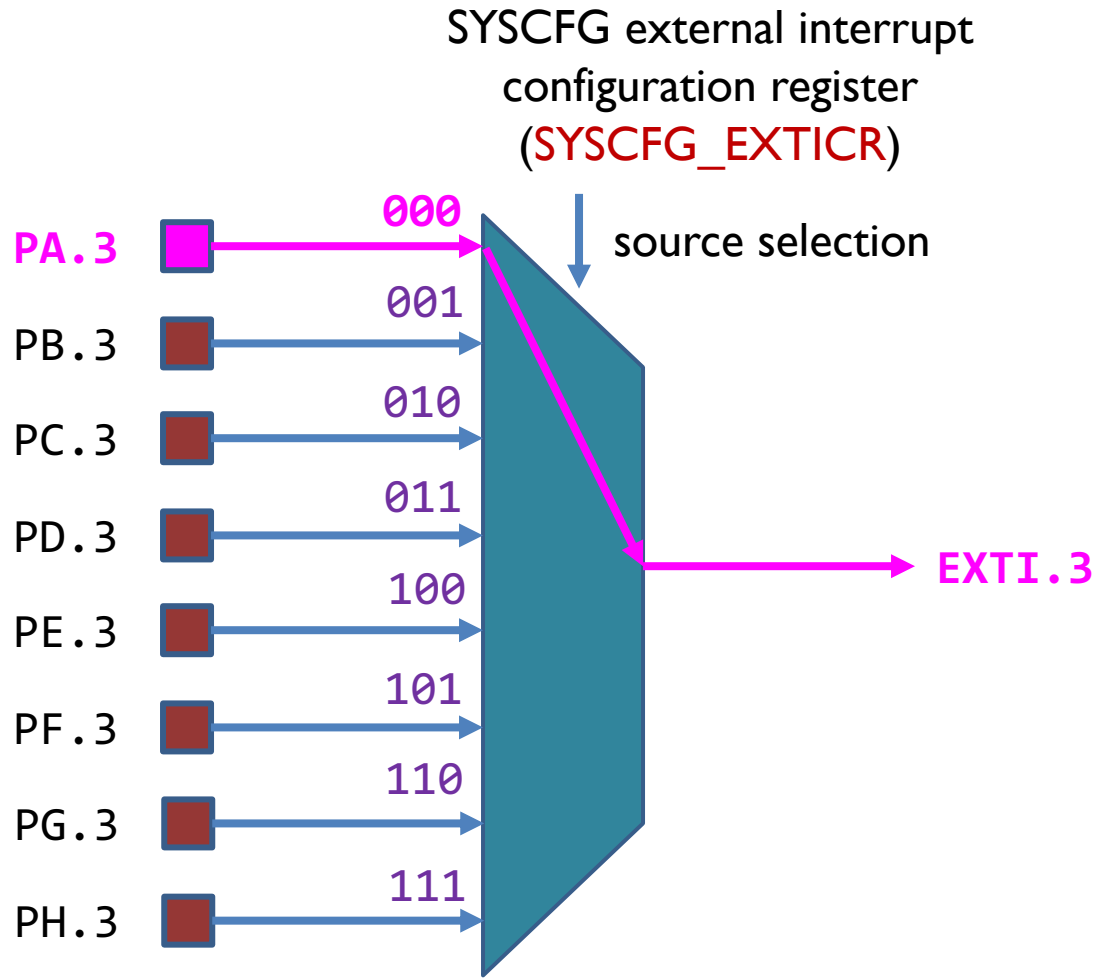
```
//  Pull PA.3 down internally; Trigger on rising edge
int main(void) {
  // Enable GPIO Clock
  RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;
  // GPIO Mode: Input(00), Output (01),
  //            AF(10),    Analog (11)
  GPIOA->MODER &= ~3U << 6;
  // GPIO Push-Pull: No pull-up, pull-down (00),
  //  Pull-up (01), Pull-down (10), Reserved (11)
  GPIOA->PUPDR &= ~3U << 6;
  GPIOA->PUPDR |= 2U << 6;     // Pull down

  NVIC_EnableIRQ(EXTI3_IRQn); // Enable Interrupt

  // Connect External Line to the GPI
  RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;
  SYSCFG->EXTICR[0] &= ~SYSCFG_EXTICR1_EXTI3;
  SYSCFG->EXTICR[0] |=  SYSCFG_EXTICR1_EXTI3_PA;




}
```



```
//  Pull PA.3 down internally; Trigger on rising edge
int main(void) {
  // Enable GPIO Clock
  RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;
  // GPIO Mode: Input(00), Output (01),
  //            AF(10),    Analog (11)
  GPIOA->MODER &= ~3U << 6;
  // GPIO Push-Pull: No pull-up, pull-down (00),
  //  Pull-up (01), Pull-down (10), Reserved (11)
  GPIOA->PUPDR &= ~3U << 6;
  GPIOA->PUPDR |= 2U << 6;    // Pull down

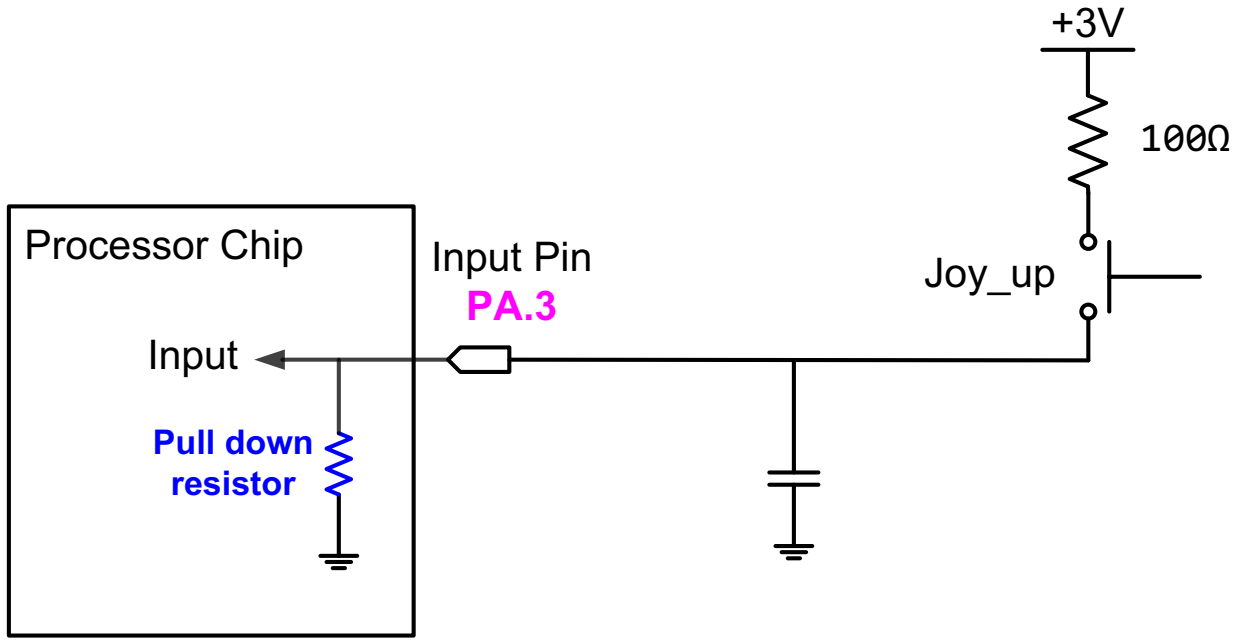
  NVIC_EnableIRQ(EXTI3_IRQn); // Enable Interrupt

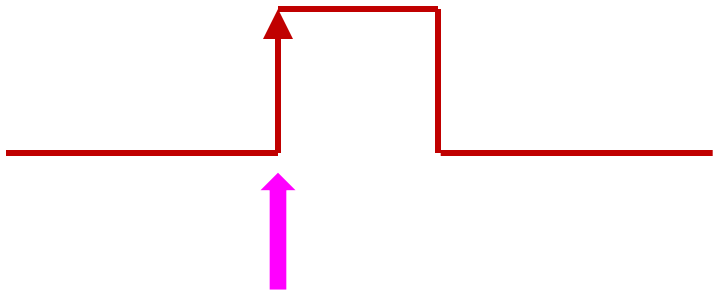
  // Connect External Line to the GPI
  RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;
  SYSCFG->EXTICR[0] &= ~SYSCFG_EXTICR1_EXTI3;
  SYSCFG->EXTICR[0] |=  SYSCFG_EXTICR1_EXTI3_PA;

  // Rising trigger selection
  // 0 = trigger disabled, 1 = trigger enabled
  EXTI->RTSR1 |= EXTI_RTSR1_RT3;



}
```





Rising edge triggers EXTI interrupt.

```
//  Pull PA.3 down internally; Trigger on rising edge
int main(void) {
  // Enable GPIO Clock
  RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;
  // GPIO Mode: Input(00), Output (01),
  //            AF(10),    Analog (11)
  GPIOA->MODER &= ~3U << 6;
  // GPIO Push-Pull: No pull-up, pull-down (00),
  //  Pull-up (01), Pull-down (10), Reserved (11)
  GPIOA->PUPDR &= ~3U << 6;
  GPIOA->PUPDR |= 2U << 6;    // Pull down

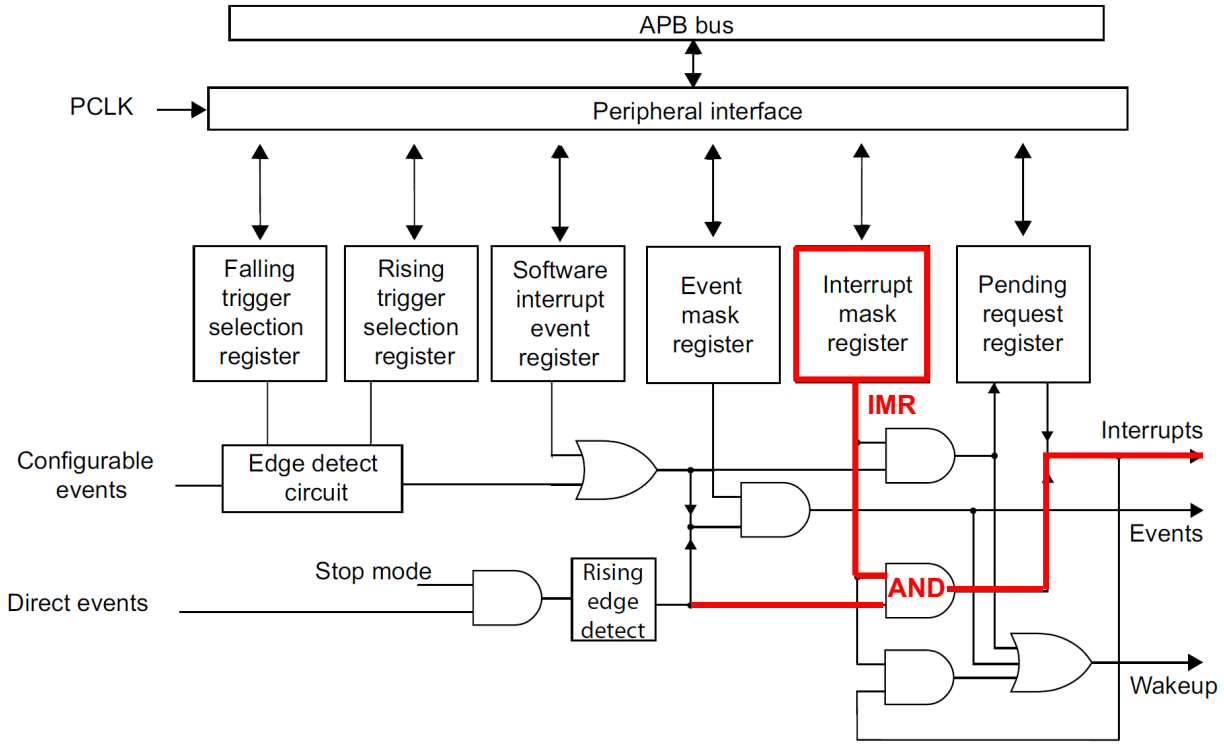
  NVIC_EnableIRQ(EXTI3_IRQn); // Enable Interrupt

  // Connect External Line to the GPI
  RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;
  SYSCFG->EXTICR[0] &= ~SYSCFG_EXTICR1_EXTI3;
  SYSCFG->EXTICR[0] |=  SYSCFG_EXTICR1_EXTI3_PA;

  // Rising trigger selection
  // 0 = trigger disabled, 1 = trigger enabled
  EXTI->RTSR1 |= EXTI_RTSR1_RT3;

  // Interrupt Mask Register
  // 0 = masked, 1 = not masked (enabled)
  EXTI->IMR1  |= EXTI_IMR1_IM3;


}
```



```
//  Pull PA.3 down internally; Trigger on rising edge
int main(void) {
  // Enable GPIO Clock
  RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;
  // GPIO Mode: Input(00), Output (01),
  //            AF(10),    Analog (11)
  GPIOA->MODER &= ~3U << 6;
  // GPIO Push-Pull: No pull-up, pull-down (00),
  //  Pull-up (01), Pull-down (10), Reserved (11)
  GPIOA->PUPDR &= ~3U << 6;
  GPIOA->PUPDR |= 2U << 6;    // Pull down

  NVIC_EnableIRQ(EXTI3_IRQn); // Enable Interrupt

  // Connect External Line to the GPI
  RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;
  SYSCFG->EXTICR[0] &= ~SYSCFG_EXTICR1_EXTI3;
  SYSCFG->EXTICR[0] |=  SYSCFG_EXTICR1_EXTI3_PA;

  // Interrupt Mask Register
  // 0 = marked, 1 = not masked (enabled)
  EXTI->IMR1  |= EXTI_IMR1_IM3;

  // Rising trigger selection
  // 0 = trigger disabled, 1 = trigger enabled
  EXTI->RTSR1 |= EXTI_RTSR1_RT3;

  while(1);
}
```

Processor runs a dead loop forever!



Interrupt Service Routine (ISR)

```
void EXTI3_IRQHandler(void) {

  // Check for EXTI 3 interrupt pending flag
  if ((EXTI->PR1 & EXTI_PR1_PIF3) != 0) {

    // Toggle LED
    GPIOB->ODR ^= 1<<8; // Toggle PB.8 output

    // Cleared interrupt pending flag by writing 1
    EXTI->PR1 |= EXTI_PR1_PIF3;
  }
}
```

